

GCA — Gestão de Código Assistida

Documento Técnico Completo para Construção da Aplicação

Versão consolidada para uso pelo Claude Code

Campo	Valor
Produto	GCA — Gestão de Código Assistida
Base documental	Somente arquivos fornecidos pelo usuário + ajustes explícitos desta conversa
Uso pretendido	Construção integral da aplicação
Restrições	Não incluir senhas, tokens, chaves, segredos ou credenciais em texto claro
Observação crítica	Menções a GPD foram descartadas; o modelo alvo é exclusivamente o GCA

1. Escopo deste documento

Este documento consolida a visão funcional, técnica, operacional, de segurança e de qualidade do GCA com foco em orientar a construção total da aplicação.

O texto foi estruturado para ser consumido por ferramenta de implementação assistida, preservando regras de governança, multi-tenancy, OCG, onboarding controlado, integração com repositórios, Code Generator, QA Readiness, Documentação Viva e Auditoria Global.

Quando havia material com credenciais ou exemplos sensíveis nos anexos, tais valores foram deliberadamente excluídos. O documento mantém apenas o comportamento esperado, campos lógicos e políticas de armazenamento/uso.

2. Premissas consolidadas e ajustes mandatórios

- O produto correto é GCA — Gestão de Código Assistida.
- O tenant não cria livremente um projeto ativo. O tenant solicita a criação do projeto e a área administrativa do GCA consolida, avalia e libera o uso.
- Após a liberação do projeto, o GP do tenant responde um questionário técnico orientado pelo stack e pelo tipo de entrega pretendida.
- O resultado do questionário técnico define exatamente quais parâmetros e integrações o tenant deverá configurar no seu próprio contexto de projeto.
- Se o projeto for melhoria ou evolução sobre sistema existente, o repositório de origem é obrigatório e o tenant deve fornecer credenciais read-only quando a análise controlada do legado for necessária.
- A documentação gerada ou consolidada pelo GCA deve ser publicada no contexto e, quando configurado, no repositório do projeto conforme estratégia aprovada.

3. Visão geral do produto

O GCA é uma meta-plataforma de orquestração, governança e visibilidade de projetos de software. Cada projeto opera com autonomia de stack, equipe, credenciais, repositório e regras específicas, mas sob governança central do GCA. O OCG é a peça central e deve ser lido antes de qualquer operação relevante.

3.1 Objetivos estratégicos

- Orquestrar múltiplos projetos com isolamento de contexto, credenciais, custos e trilhas.
- Controlar o ciclo documental em 7 pilares antes de liberar automações sensíveis.
- Permitir geração assistida de código e documentação com revisão humana obrigatória nos pontos críticos.

- Executar testes em ambiente isolado por projeto com rastreabilidade de evidências.
- Consolidar visão administrativa global sem quebrar o isolamento entre tenants.

3.2 Público-alvo e papéis

Perfil	Escopo	Responsabilidade principal
Admin do GCA	Global	Cria e libera projetos, designa GPs, configura parâmetros globais e políticas transversais.
Gerente de Projeto (GP)	Projeto	Configura OCG, responde questionário técnico, mantém equipe, credenciais e pendências.
Tech Lead	Projeto	Define stack, repositório, critérios técnicos, aprova código e override documentado do Gatekeeper.
Desenvolvedor Sênior	Projeto	Aciona gerações, revisa artefatos técnicos e aprova quando autorizado.
Desenvolvedor Pleno	Projeto	Contribui sob fluxo de aprovação e permissões do projeto.
QA Engineer	Projeto	Mantém plano de testes, aprova evidências e valida prontidão.
Compliance / Segurança	Global ou projeto	Monitora quarentena, auditoria e aderência regulatória sem acesso a segredos.
Stakeholder / Gestão	Leitura	Acompanha dashboards e relatórios sem alteração operacional.

4. Arquitetura conceitual

4.1 Diagrama de camadas

```

flowchart TB
    subgraph GCA_Global["Camada Global do GCA"]
        A1["Autenticação Global"]
        A2["RBAC Global"]
        A3["Gestão de Credenciais Globais"]
        A4["Observabilidade e Auditoria Global"]
        A5["Administração / Consolidação de Projetos"]
    end
  
```

```

end

subgraph TenantA["Tenant / Projeto A"]
  B1["OCG"]
  B2["Módulos M1-M14"]
end

subgraph TenantB["Tenant / Projeto B"]
  C1["OCG"]
  C2["Módulos M1-M14"]
end

subgraph Infra["Infraestrutura Compartilhada"]
  D1["PostgreSQL por schema"]
  D2["Redis por namespace"]
  D3["Kafka por tópico"]
  D4["Storage por prefixo"]
  D5["Containers efêmeros de QA"]
end

GCA_Global --> TenantA
GCA_Global --> TenantB
TenantA --> Infra
TenantB --> Infra

```

A arquitetura é formada por uma camada global do GCA, tenants isolados por projeto e infraestrutura compartilhada com segregação por schema, namespace, tópico e prefixo de storage.

4.2 Stack técnico consolidado

Camada	Tecnologias / padrões
Backend	Python + FastAPI + SQLAlchemy async + asyncpg + Alembic
Frontend	React 18 + TypeScript + Vite + Tailwind CSS + Zustand + React Query + React Router + React Hook Form + Zod
Banco de dados	PostgreSQL 16 com schema isolado por projeto
Cache e sessão	Redis 7 com namespace por projeto e store de sessão
Mensageria	Apache Kafka
Orquestração	n8n opcional, conforme cenário
Autenticação	JWT RS256 com controle central de sessão e bcrypt
E-mail	SMTP configurável
Execução de testes	Containers Docker efêmeros por execução
Deploy	Docker Compose e evolução para Kubernetes conforme maturidade

5. Modelo de onboarding administrativo

5.1 Regra obrigatória de criação de projeto

A criação operacional de um projeto ativo deve passar pela área administrativa do GCA. O tenant solicita a criação; o Admin consolida os dados iniciais, avalia pré-requisitos e libera o uso do tenant.

```
sequenceDiagram
    participant Tenant as Tenant / Solicitante
    participant Admin as Admin do GCA
    participant GCA as GCA
    participant GP as Gerente de Projeto

    Tenant->>Admin: Solicita criação do projeto
    Admin->>GCA: Registra solicitação e dados iniciais
    GCA-->>Admin: Valida campos mínimos e status draft
    Admin->>GCA: Aprova liberação inicial
    GCA-->>GP: Libera acesso controlado ao tenant
    GP->>GCA: Inicia questionário técnico orientado por stack
    GCA-->>GP: Lista parametrizações obrigatórias do projeto
```

5.2 Etapas do onboarding

Etapa	Responsável	Descrição
Solicitação de projeto	Tenant	Solicita abertura do projeto e informa dados mínimos de negócio.
Consolidação administrativa	Admin do GCA	Cria registro inicial, confere escopo e libera o tenant para parametrização.
Questionário técnico	GP do tenant	Responde perguntas orientadas por stack, tipo de saída e contexto legado.
Parametrização do tenant	GP / Tech Lead	Configura IA, repositório, integrações, banco, frontend, backend, infraestrutura e políticas.
Provisionamento	GCA	Cria schema, namespace, tópicos, storage e webhooks quando aplicável.
Ativação	GCA + Admin	Projeto assume estado ativo somente após conclusão válida do provisioning.

5.3 Questionário técnico orientado por stack

O questionário técnico não é apenas informativo. Ele deve dirigir a parametrização obrigatória do projeto no tenant e definir entregáveis, restrições e integrações necessárias.

- Tipo de saída pretendida: executável/desktop, web app, API, mobile, melhoria em sistema existente ou nova funcionalidade em sistema existente.
- Stack principal: linguagem, frameworks, front-end, back-end, banco de dados, mensageria, storage e requisitos de cloud.
- Estratégia de repositório: URL, branch principal, branch de documentação, política de merge e forma de publicação documental.
- IA do projeto: provedor, modelo, limites, política de envio externo, retenção, fallback e mascaramento conforme compliance.
- Integrações do projeto: n8n, Jira, Figma, Trello, Slack, Teams, SMTP, storage, Kafka e outras integrações necessárias.
- Contexto de legado: existência de sistema prévio, escopo de análise controlada, necessidade de credenciais read-only e área do código a ser analisada.
- Entregáveis esperados: código, documentação negocial, documentação técnica, testes, evidências de QA, PRs e publicações documentais.

5.4 Perfis de saída e implicações

OutputProfile	Implicações mínimas
Executável / Desktop	Deve declarar empacotamento, runtime, instalador, política de atualização e eventual repositório.
Web App / API	Deve declarar frontend/backend, deploy, segurança, observabilidade e repositório.
Melhoria em sistema existente	Repositório obrigatório; análise controlada do legado e documentação consolidada no contexto do projeto.
Nova funcionalidade em sistema existente	Repositório obrigatório; credenciais read-only quando necessário; mapeamento de impacto e documentação viva.

6. OCG — Objeto de Contexto Global

Nenhum módulo opera sobre um projeto sem antes ler seu OCG. O contexto precede a ação, e mudanças relevantes propagam automaticamente para os módulos dependentes.

Dimensão	Conteúdo resumido	Mutabilidade
Identidade	Nome, slug, descrição, metodologia, perfis e status do projeto.	GP com trilha
Credenciais de IA	Provedor ativo, modelo, limites e política de envio externo.	GP/Admin
Repositório	URL, branches, estratégia de merge, estado do webhook.	Tech Lead / GP
Integrações	Status, responsáveis, expiração, escopo e disponibilidade.	GP/Admin
Stack e infraestrutura	Linguagens, frameworks, banco, mensageria e cloud.	Tech Lead com justificativa
ComplianceProfile	Regras de uso de IA, retenção, mascaramento e trilhas.	GP com validação
Artefatos ativos	Documentos, versões, hashes, status e vínculos.	Automático
QA seleção	Tipos de teste habilitados e exigências mínimas.	QA / Tech Lead
Equipe e RBAC	Membros, papéis e permissões especiais.	GP/Admin
Estado de entrega	Fase, score Gatekeeper, status QA, hashes e pendências.	Automático
Histórico	Log append-only de mudanças estruturais.	Imutável logicamente

6.1 Propagação automática do OCG

flowchart TD

```

A["Mudança no OCG"] --> B["Regras de propagação"]
B --> C["Reavaliar ComplianceProfile"]
B --> D["Atualizar templates de geração"]
B --> E["Recalcular plano QA"]
B --> F["Atualizar Documentação Viva"]
C --> G["Restringir provedores de IA e mascaramento"]
D --> H["Refazer prompts e parâmetros técnicos"]
E --> I["Trocar imagem base, testes e políticas"]
F --> J["Regerar documentação técnica/negocial"]

```

7. Módulos funcionais

Módulo	Responsabilidade principal	OCG consumido
M1 — Autenticação e Sessões	Identidade, login, tokens, revogação e bloqueio.	Não usa OCG de projeto
M2 — Gestão de Usuários	Usuários, convites, RBAC e papéis.	Não usa OCG de projeto
M3 — OCG Wizard	Criação sequencial do projeto, validações e provisionamento.	Cria o OCG
M4 — Ingestão	Upload, extração, pré-triagem local, classificação e quarentena.	ComplianceProfile, IA, artefatos
M5 — Gatekeeper	Avaliação dos 7 pilares e scoring formal.	Artefatos consolidados, compliance, estado
M6 — Merge Engine	Documento mestre, resolução de conflitos e consolidação.	Artefatos ativos e histórico
M7 — Arguidor Técnico	Perguntas dirigidas, entregáveis esperados e retorno ao ciclo.	Resultado do Gatekeeper e conflitos
M8 — Code Generator	Geração assistida de artefatos técnicos e código.	Stack, repositório, IA do projeto, estado
M9 — QA Readiness	Planeja e executa testes com isolamento por projeto.	QA seleção, stack, repositório, compliance
M10 — Análise de Legado	Leitura controlada de sistemas ou repositórios legados.	Credenciais e mapa de integrações
M11 — Roadmap e Histórico	Evolução de requisitos, decisões e vínculos.	Artefatos, estados e trilhas
M12 — Documentação Viva	Regenera e publica documentação técnica do projeto.	Stack, repositório, IA do projeto
M13 — Dashboard Executivo	Visão consolidada de progresso, riscos e indicadores.	OCGs consolidados em leitura
M14 — Auditoria Global	Registro e prova de integridade das trilhas.	Histórico e eventos de todos os projetos

7.1 Sequência operacional consolidada

flowchart TD

```

A["M1/M2 - Governança base"] --> B["M3 - OCG Wizard"]
B --> C["M4 - Ingestão de artefatos"]
C --> D["M6 - Merge Engine"]
D --> E["M5 - Gatekeeper"]

```



```

E -->|Blocker| F["M7 - Arguidor Técnico"]
F --> D
E -->|Aprovado| G["M8 - Code Generator"]
G --> H["M9 - QA Readiness"]
H --> I["M10 - Executor isolado / Legado conforme escopo"]
I --> J["M11 - Roadmap e Histórico"]
J --> K["M12 - Documentação Viva"]
K --> L["M13 - Dashboard Executivo"]
L --> M["M14 - Auditoria Global"]

```

8. Regras de ingestão, merge e gatekeeping

8.1 Ingestão e pré-triagem local

- Upload suportado para PDF, DOCX, TXT e Markdown.
- Extração local do texto com hash SHA-256 e tratamento de duplicidade antes da IA.
- Pré-triagem local obrigatória de PII com regex, dicionários e regras do Admin.
- Se o ComplianceProfile exigir, o texto deve ser mascarado ou resumido antes de qualquer envio a provedor externo.
- Artefatos com indício de PII não autorizado entram em quarentena e ficam indisponíveis para merge até decisão formal.

8.2 Quarentena LGPD

Etapa	Descrição
Entrada em quarentena	Artefato recebe estado pii_quarantine e não segue para merge.
Análise	Compliance ou capability APPROVE_QUARANTINE decide rejeitar, mascarar, aprovar ou devolver.
Versão saneada	Se houver mascaramento, cria-se nova versão vinculada ao original.
Reprocessamento	A versão saneada volta à classificação e segue o fluxo normal.
Trilha	Decisão, justificativa, autor e data ficam em auditoria.

8.3 Merge, Gatekeeper e Arguidor Técnico

```

sequenceDiagram
    participant Ingestao as M4 Ingestão
    participant Merge as M6 Merge Engine
    participant GK as M5 Gatekeeper
    participant Arg as M7 Arguidor Técnico

```

participant Projeto as GP / Tech Lead

```

Ingestao->>Merge: Artefatos extraídos e classificados
Merge->>GK: Documento mestre + conflitos + mapa de origem
GK->>Projeto: Score, blockers, riscos e recomendação
GK->>Arg: Dispara perguntas quando houver gaps
Arg->>Projeto: Solicita respostas e evidências
Projeto->>Arg: Responde e anexa material
Arg->>Merge: Reabre ciclo com novos artefatos versionados

```

8.4 Regras de resolução de conflito

- Informação mais específica prevalece sobre informação genérica quando não houver contradição.
- Conflito explícito entre artefatos do mesmo nível gera pendência obrigatória ao Gatekeeper.
- Decisões registradas, ADRs aprovadas ou respostas formais do Arguidor prevalecem sobre rascunhos anteriores.
- Requisito aprovado e mais recente prevalece sobre versões anteriores, preservando vínculo histórico.
- Nenhum conflito crítico pode ser resolvido silenciosamente.

9. Code Generator, repositório e legado

9.1 Regras do Code Generator

Etapa	Detalhe
Acionamento	Tech Lead, Dev Sênior ou papel autorizado solicita geração.
Leitura do contexto	OCG, artefatos aprovados, stack, output profile, compliance e padrões do projeto.
Montagem do prompt	Contexto + instrução técnica; segredo da IA somente no canal seguro da integração.
Geração	Saída entra em drafting ou in_review com UUID de rastreabilidade.
Revisão humana	Aprovação, rejeição ou ajuste por papel autorizador.
Push / publicação	Somente com credenciais do repositório do projeto e estratégia aprovada.
Trilha	Commit, PR ou rejeição ficam vinculados ao requisito, arquivo e autor da ação.

9.2 Fluxo de análise controlada do legado

Quando o projeto representar melhoria ou nova funcionalidade sobre sistema existente, o repositório de origem passa a ser obrigatório. Se a estratégia envolver leitura automatizada por IA, o tenant deve fornecer credenciais read-only específicas para análise controlada.

```
sequenceDiagram
    participant GP as GP do Tenant
    participant GCA as GCA
    participant Repo as Repositório do Sistema Existente
    participant IA as IA do Projeto
    participant Docs as Documentação Viva

    GP->>GCA: Informa repositório e escopo de atuação
    GCA->>Repo: Valida conectividade com credencial read-only
    Repo-->>GCA: Estrutura e conteúdo autorizados
    GCA->>IA: Envia contexto permitido conforme compliance
    IA-->>GCA: Análise do código existente e insumos documentais
    GCA->>Docs: Consolida documentação comercial e técnica
    Docs-->>GP: Publica no contexto do tenant e, se configurado, no repositório
```

9.3 Webhooks e publicação documental

- GitHub, GitLab e Bitbucket podem registrar webhooks automaticamente quando o token permitir.
- Após cadastro ou rotação de segredo, o GCA executa teste de conectividade por evento assinado ou desafio equivalente.
- O webhook só assume estado active após resposta válida, assinatura conferida e persistência do resultado em auditoria.
- A Documentação Viva pode ser acionada por aprovação interna, por evento do repositório ou por webhook de push/merge aprovado.
- A publicação documental pode ocorrer por commit direto ou Pull Request automático.

10. Segurança, credenciais e conformidade

10.1 Princípios mandatórios

- Segredos armazenados com criptografia forte em repouso.
- Nenhum papel funcional visualiza o valor bruto do segredo após o cadastro.
- Todo uso operacional de segredo descriptografado gera evento de auditoria.
- Há isolamento entre credenciais do GCA e credenciais dos projetos.
- HTTPS/TLS, hashing de senha com bcrypt, validação de entrada e logging sem segredos são mandatórios.

10.2 Fluxo de rotação e monitoração

flowchart TD

```

A["Cadastrar nova credencial"] --> B["Validar antes da troca"]
B --> C["Trocar ponteiro operacional"]
C --> D["Encerrar chamadas antigas com segurança"]
D --> E["Invalidar ou expirar credencial anterior"]
E --> F["Registrar auditoria da rotação"]

G["Eventos monitorados"] --> H["Sem crédito / Expiração / Suspeita / Falhas repetidas"]
H --> I["Circuit breaker, alertas e degradação controlada"]

```

10.3 Controles de conformidade

Controle	Descrição
OWASP Top 10	Proteções esperadas no backend, frontend e integrações.
LGPD / GDPR-ready	Tratamento de dados, quarentena, mascaramento e trilhas.
NIST aligned	Referência para postura de segurança e governança.
RBAC mínimo	Permissões por papel e capability especial para quarentena.
Auditoria encadeada	Eventos com prova de integridade e rastreabilidade.

11. Multi-tenancy, dados e provisionamento

11.1 Isolamento por tenant

Camada	Mecanismo	Detalhe
PostgreSQL	Schema por projeto	proj_{slug} com ownership e políticas segregadas.
Redis	Namespace por projeto e sessão	Prefixos por project_id e stores separados por finalidade.
Kafka	Tópicos prefixados	gca.{project_id}.{tipo}.
Storage	Diretório/prefixo isolado	Sem compartilhamento entre projetos.
IA	Credenciais e contexto isolados	Sem reuso de prompts ou memória entre projetos.

Camada	Mecanismo	Detalhe
Executor QA	Container efêmero por execução	Sem estado compartilhado e com política de rede.

11.2 Provisionamento compensatório

- Criar schema do projeto no PostgreSQL.
- Reservar namespace no Redis.
- Criar ou registrar tópicos Kafka quando aplicável.
- Criar diretório ou prefixo de storage do projeto.
- Registrar webhook automático quando o provedor suportar; caso contrário, abrir pendência manual obrigatória.
- Se qualquer etapa falhar, executar compensações idempotentes e marcar o projeto como `provisioning_failed`.

11.3 Estados principais

Objeto	Estados principais
Projeto	draft, provisioning, provisioning_failed, active, degraded, suspended, archived
Artefato	uploaded, extracted, pii_quarantine, classified, pending_review, merged, rejected, superseded
Gatekeeper	not_started, in_progress, blocked, approved, approved_with_override
Geração de código	requested, drafting, in_review, approved, rejected, pushed, failed
Execução de teste	queued, running, passed, failed, blocked, waived
Webhook	pending_setup, active, invalid, suspended
Credencial	valid, warning, expired, suspected, rotated, disabled

12. Estratégia de testes e qualidade

Os anexos indicam testes unitários, integrados, regressivos, E2E, UAT e execução isolada por stack. Para orientar a construção integral da aplicação, esta seção expande a taxonomia de QA em linha com a solicitação do usuário e com boas práticas de engenharia.

Categoria	Objetivo principal	Aplicação no GCA
Smoke / Sanity	Verificar o essencial após build ou deploy.	Health checks, login, rotas críticas e abertura do dashboard.
Unitários	Validar funções e classes isoladas.	Services, regras de negócio, segurança, validadores, score do Gatekeeper.
Integração	Validar integração entre componentes.	FastAPI + PostgreSQL + Redis + Kafka + SMTP + storage.
Contrato / API	Garantir aderência entre consumidores e provedores.	OpenAPI, schemas, payloads, erros padronizados e versionamento.
Repositório / Webhook	Validar conectividade e eventos.	Registro, assinatura, rotação e consumo de webhooks.
Regressão	Evitar reintrodução de defeitos.	Fluxos administrativos, OCG Wizard, auditoria e publicação documental.
E2E	Validar a jornada ponta a ponta.	Solicitação de projeto, liberação, parametrização, ingestão e QA.
UAT	Validar aderência ao processo do usuário.	Admin, GP, Tech Lead e QA executando cenários reais.
Segurança	Detectar falhas técnicas e de postura.	Auth, RBAC, JWT, CORS, secrets, trilhas e políticas de acesso.
SAST / SCA	Analisar código e dependências.	Pipeline de backend, frontend e imagens Docker.
DAST	Testar aplicação em execução.	Endpoints expostos, autenticação e sessão.
Performance / Carga	Medir throughput e latência.	API, dashboard, ingestão e filas.
Stress / Soak	Avaliar limite e estabilidade prolongada.	Execuções simultâneas, filas e workers.
Resiliência / Recuperação	Validar degradação controlada e retomada.	Falha de credenciais, repositório, banco, cache e webhooks.
Backup / Restore	Comprovar recuperação operacional.	OCGHistory, auditoria e banco.
Acessibilidade	Assegurar uso adequado da interface.	Navegação por teclado, contraste, rótulos e feedbacks.
Compatibilidade	Reduzir variação por ambiente.	Navegadores, tamanhos de tela e stacks de execução.

Categoria	Objetivo principal	Aplicação no GCA
Observabilidade	Confirmar logs, métricas e alertas.	Eventos, correlação, métricas de health e SLAs internos.

12.1 Fluxo de QA Readiness e executor isolado

```
sequenceDiagram
    participant QA as QA Readiness
    participant Repo as Repositório do Projeto
    participant Exec as Executor Isolado
    participant Store as Storage / Evidências
    participant Dash as Dashboard

    QA->>Repo: Clona repositório com credenciais do projeto
    QA->>Exec: Cria container efêmero conforme stack
    Exec->>Exec: Instala dependências e executa plano de testes
    Exec->>Store: Persiste logs, cobertura e evidências
    Exec-->>QA: Retorna status passed/failed/blocked/waived
    QA-->>Dash: Atualiza indicadores e rastreabilidade
```

12.2 Seleção de imagem base por stack

Stack / perfil	Imagem base padrão	Observação
Python	python:3.12-slim	pytest, scripts e serviços Python
Node.js / Frontend	node:20-bullseye	Jest, Vitest, builds e E2E; variantes com browser quando exigido
Java	eclipse-temurin:21-jdk	JUnit, Maven ou Gradle
Go	golang:1.22-alpine	go test e build efêmero
.NET	mcr.microsoft.com/dotnet/sdk:8.0	testes .NET e builds temporários
Imagem customizada	registry/{projeto}/qa-base:{tag}	Permitida quando declarada e versionada no OCG

13. Observabilidade, operação e produção

- Health checks do backend e disponibilidade do frontend devem ser expostos e monitorados.
- Logs estruturados sem segredos são mandatórios.
- Alertas de sistema, histórico de alertas, tickets e métricas executivas devem alimentar o dashboard administrativo.

- Backup, restore, revisão de performance e atualização de segurança devem constar do plano operacional.

13.1 Métricas e indicadores mínimos

Indicador	Descrição
Uptime	Disponibilidade consolidada do backend e serviços dependentes.
Tempo médio de resposta	Latência média da API e dos fluxos críticos.
Success rate	Taxa de sucesso das operações administrativas e de projeto.
Score Gatekeeper	Pontuação por pilar e score geral.
Status de testes	Quantidade e proporção de passed, failed, blocked e waived.
Saúde das credenciais	valid, warning, expired, suspected, rotated, disabled.
Saúde dos webhooks	pending_setup, active, invalid, suspended.
Custos e uso de IA	Tokens, custo estimado e limites por projeto.

14. Estrutura recomendada de implementação

```

GCA/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   └── constants.py
│   │   ├── db/
│   │   │   ├── database.py
│   │   │   └── migrations/
│   │   ├── models/
│   │   ├── routers/
│   │   ├── services/
│   │   ├── middleware/
│   │   └── tests/
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── stores/
│   │   ├── lib/
│   │   └── App.tsx

```



```
| └─ Dockerfile
└─ docker-compose.yml
```

15. Requisitos normativos para o Claude Code

- Não persistir ou expor segredos em código, logs, payloads de erro, fixtures ou documentação.
- Implementar o fluxo administrativo de solicitação, consolidação e liberação de projetos antes do onboarding completo do tenant.
- Materializar o questionário técnico orientado por stack e por OutputProfile como parte do M3.
- Permitir análise read-only de repositório legado quando o projeto declarar melhoria/evolução em sistema existente.
- Persistir toda decisão relevante em trilha auditável e encadeada.
- Ler o OCG antes de qualquer operação relevante de módulo.
- Aplicar isolamento rigoroso entre tenants em banco, cache, storage, mensageria, IA, QA e webhooks.
- Implementar os módulos em ordem evolutiva, começando por governança base, OCG Wizard, ingestão, merge/gatekeeper e somente depois geração assistida mais avançada.

16. Roadmap técnico sugerido para construção

Fase	Entregas principais
Fase 1	Autenticação, usuários, RBAC, bootstrap administrativo, health checks.
Fase 2	Solicitação e consolidação administrativa de projetos, OCG Wizard, provisionamento compensatório.
Fase 3	Ingestão de artefatos, pré-triagem local, quarentena LGPD, merge e Gatekeeper.
Fase 4	Arguidor Técnico, publicação de respostas versionadas e reavaliação.
Fase 5	Code Generator com revisão humana e integração controlada ao repositório.
Fase 6	QA Readiness, executor isolado, imagens por stack, evidências e dashboard.
Fase 7	Documentação Viva, roadmap, auditoria global, alertas e operação.

17. Diagramas complementares

17.1 Estados do projeto

```
stateDiagram-v2
    [*] --> draft
    draft --> provisioning
    provisioning --> provisioning_failed
    provisioning --> active
    active --> degraded
    degraded --> active
    active --> suspended
    suspended --> active
    active --> archived
    degraded --> archived
    suspended --> archived
```

17.2 Publicação de documentação viva

```
flowchart LR
    A["Código aprovado / Push / Merge"] --> B["Disparo M12"]
    B --> C["Leitura do OCG"]
    C --> D["Geração / Consolidação documental"]
    D --> E["Estratégia de publicação"]
    E -->|Direta| F["Commit em branch/pasta de docs"]
    E -->|PR| G["Branch docs/gca-{uuid} + Pull Request"]
```

18. Encerramento

Este documento foi consolidado para servir como base de implementação do GCA no contexto de construção assistida. Ele já incorpora a substituição explícita do modelo alvo para GCA, a regra de solicitação/liberação administrativa do projeto, o questionário técnico pós-liberação e a obrigatoriedade de análise controlada do legado quando o projeto atuar sobre sistemas existentes.

Na próxima revisão, o documento pode ser refinado com mais restrições, exclusões, detalhamento de APIs, modelo de dados físico, contratos OpenAPI e critérios de aceite por módulo.

ANEXO A - QUESTIONÁRIO TÉCNICO EXTERNO DO TENANT

Este anexo define o questionário técnico a ser respondido externamente pelo tenant, fora do fluxo interno de parametrização, para posterior ingestão no início do projeto. As respostas, os resultados, os gaps detectados, as sugestões do n8n do GCA e o status do questionário são válidos exclusivamente para o projeto correspondente e não podem ser reutilizados por outros tenants.

Regras mandatórias do anexo

- O tenant deve responder o questionário antes da parametrização final do projeto.
- O questionário deve ser compartimentalizado por tenant e por projeto.
- O n8n do GCA deve analisar coerência, conflitos, lacunas e dependências não respondidas.
- O questionário deve poder ser exportado e apresentado em página HTML externa para coleta e validação pelas áreas envolvidas.
- Somente integrantes autorizados do projeto e Admin do GCA podem acessar o conteúdo do questionário, seus resultados e sugestões.
- Os logs de todos os projetos devem permanecer consolidados na área administrativa do GCA para auditoria, coaching e investigação de incidentes.

A.1 Identificação e enquadramento inicial

Pergunta / Campo	Resposta por seleção
1. Nome do projeto	_____
2. Slug do projeto	_____
3. O projeto é direcionado a fazer melhorias ou alterar um projeto já existente?	☐ Sim ☐ Não
4. Tipo de iniciativa	☐ Novo sistema ☐ Melhoria em sistema existente ☐ Nova funcionalidade em sistema existente ☐ Refatoração técnica ☐ Modernização/Migração ☐ Integração ☐ Automação interna ☐ POC/MVP
5. Criticidade	☐ Baixa ☐ Média ☐ Alta ☐ Crítica
6. Classificação da informação	☐ Pública

	☐ Interna ☐ Confidencial ☐ Restrita
--	--

A.2 Bloco obrigatório para projetos existentes

Pergunta / Campo	Resposta por seleção
7. Nome do sistema existente	_____
8. Repositório principal do sistema existente	_____
9. Repositórios adicionais relacionados	_____
10. Nível de acesso fornecido ao repositório	☐ Read-only ☐ Read + metadata ☐ Read + PR ☐ Outro
11. Objetivo da alteração	☐ Correção ☐ Evolução funcional ☐ Refatoração ☐ Integração ☐ Redução de débito técnico ☐ Migração de arquitetura ☐ Adequação de segurança/compliance ☐ Performance
12. O tenant autoriza o n8n do GCA a analisar o repositório?	☐ Sim ☐ Não
13. A análise do n8n deve verificar	☐ Arquitetura atual ☐ Linguagens e frameworks ☐ Dependências e versões ☐ Itens deprecated ☐ Pipelines CI/CD ☐ Testes existentes ☐ Riscos técnicos ☐ Documentação ausente ☐ Integrações detectadas
14. Relatório técnico esperado para o GP	☐ Resumo executivo ☐ Arquitetura identificada ☐ Stack detectada ☐ Riscos técnicos ☐ Sugestão de backlog ☐ Sugestão de modernização ☐ Lacunas de testes ☐ Lacunas de documentação

A.3 Perfil de entrega e arquitetura alvo

Pergunta / Campo	Resposta por seleção
15. Entregável principal do projeto	☐ Executável desktop ☐ Aplicação web ☐ API ☐ Microserviço ☐ Aplicativo mobile ☐ Dashboard ☐ Job/Worker ☐ CLI ☐ Biblioteca/SDK
16. Perfil arquitetural preferido	☐ Monólito ☐ Monólito modular ☐ Microserviços ☐ Event-driven ☐ Hexagonal ☐ Clean Architecture ☐ Serverless ☐ Desktop local
17. Modelo de execução	☐ Stand-alone ☐ On-premises ☐ Cloud ☐ Híbrido ☐ Containerizado ☐ Offline com sincronização posterior
18. Há necessidade de multi-tenant no sistema a ser construído?	☐ Sim ☐ Não ☐ Talvez
19. Há necessidade de alta disponibilidade?	☐ Sim ☐ Não ☐ Futuramente
20. Há necessidade de processamento assíncrono/jobs?	☐ Sim ☐ Não

A.4 Frontend

Pergunta / Campo	Resposta por seleção
21. O projeto terá frontend?	☐ Sim ☐ Não
22. Tipo de frontend	☐ Web SPA ☐ SSR

	☐ PWA ☐ Desktop UI ☐ Mobile app ☐ Painel administrativo ☐ Portal autenticado
23. Stack frontend preferida	☐ React ☐ Vue ☐ Angular ☐ Next.js ☐ Vite + React ☐ Electron ☐ Flutter ☐ React Native ☐ Sem preferência
24. Linguagem preferida	☐ TypeScript ☐ JavaScript ☐ Outra
25. Requisitos de frontend	☐ Responsividade ☐ Acessibilidade ☐ Dark theme ☐ Formulários complexos ☐ Gráficos ☐ Upload de arquivos ☐ Impressão/PDF ☐ Internacionalização

A.5 Backend e APIs

Pergunta / Campo	Resposta por seleção
26. O projeto terá backend?	☐ Sim ☐ Não
27. Linguagem backend preferida	☐ Python ☐ Node.js ☐ Java ☐ C# ☐ Go ☐ PHP ☐ Kotlin ☐ Outra
28. Framework backend preferido	☐ FastAPI ☐ Django ☐ Flask ☐ NestJS

	☐ Express ☐ Spring Boot ☐ ASP.NET ☐ Quarkus ☐ Sem preferência
29. Tipo de backend	☐ REST API ☐ GraphQL ☐ gRPC ☐ WebSocket ☐ Batch ☐ Worker ☐ BFF ☐ Misto
30. Requisitos de backend	☐ Autenticação ☐ RBAC ☐ Webhooks ☐ Jobs ☐ Auditoria ☐ Versionamento de API ☐ Rate limiting ☐ Observabilidade ☐ Integração com IA

A.6 Dados, cache, mensageria e automação

Pergunta / Campo	Resposta por seleção
31. Persistência principal	☐ PostgreSQL ☐ MySQL ☐ SQL Server ☐ Oracle ☐ MongoDB ☐ SQLite ☐ Sem preferência
32. Perfil de uso do banco	☐ Transacional ☐ Analítico ☐ Documental ☐ Catálogo ☐ Event store ☐ Misto
33. Necessidade de Redis/cache	☐ Sim ☐ Não ☐ Talvez
34. Finalidade do Redis	☐ Cache de leitura

	☐ Sessões ☐ Rate limiting ☐ Pub/Sub ☐ Locks distribuídos ☐ Filas leves
35. Necessidade de Kafka/mensageria	☐ Sim ☐ Não ☐ Talvez
36. Finalidade da mensageria	☐ Eventos de domínio ☐ Integrações assíncronas ☐ Processamento em background ☐ Orquestração entre serviços ☐ Telemetria
37. Uso de n8n	☐ Sim ☐ Não ☐ Talvez
38. Finalidade do n8n	☐ Leitura/análise de repositório legado ☐ Automação de integrações ☐ Notificações ☐ Geração de relatórios ☐ ETL ☐ Disparo de webhooks ☐ Aprovações

A.7 Inteligência artificial, segurança e observabilidade

Pergunta / Campo	Resposta por seleção
39. Uso de IA no projeto	☐ Sim ☐ Não ☐ Talvez
40. Finalidade da IA	☐ Análise de requisitos ☐ Geração de código ☐ Documentação técnica ☐ Documentação negocial ☐ Revisão de código ☐ Testes automatizados ☐ Classificação de artefatos ☐ Chat assistivo
41. Provedor preferencial	☐ Anthropic ☐ OpenAI ☐ Gemini ☐ DeepSeek ☐ Grok

	☐ Outro ☐ Sem preferência
42. Restrições para IA externa	☐ Mascaramento ☐ Anonimização ☐ Bloqueio total ☐ Envio permitido por política ☐ Avaliação por tipo de dado
43. Controles de segurança obrigatórios	☐ JWT ☐ OAuth2 ☐ SSO ☐ MFA ☐ Criptografia em trânsito ☐ Criptografia em repouso ☐ Vault de segredos ☐ Rotação de credenciais ☐ Trilhas de auditoria
44. Observabilidade exigida	☐ Logs estruturados ☐ Métricas ☐ Tracing ☐ Health checks ☐ Alertas ☐ Dashboard operacional ☐ Dashboard executivo

A.8 Testes, validação e entregáveis

Pergunta / Campo	Resposta por seleção
45. Tipos mínimos de teste	☐ Smoke ☐ Sanity ☐ Unitários ☐ Integração ☐ Contrato/API ☐ E2E ☐ UAT ☐ Regressão ☐ Segurança ☐ SAST/SCA ☐ DAST ☐ Performance/Carga ☐ Stress/Soak ☐ Resiliência/Recuperação ☐ Backup/Restore ☐ Acessibilidade ☐ Compatibilidade

46. Quality gate automatizado	☐ Sim ☐ Não
47. Evidência formal de QA	☐ Sim ☐ Não
48. Entregáveis esperados do pipeline do GCA	☐ Sugestão de arquitetura ☐ Sugestão de stack ☐ Documento técnico consolidado ☐ Documento negocial consolidado ☐ Gap analysis ☐ Backlog inicial ☐ Plano de testes ☐ Plano de segurança ☐ Plano de observabilidade ☐ Plano de deploy
49. Formato de retorno desejado	☐ Painel no GCA ☐ HTML ☐ Markdown ☐ DOCX ☐ PDF ☐ JSON estruturado ☐ YAML

A.9 Modelo mínimo da página HTML externa

```

<form id="gca-tenant-questionnaire">
  <section data-block="identificacao">
    <label>Nome do projeto</label><input type="text" name="project_name" />
    <fieldset>
      <legend>Projeto existente?</legend>
      <label><input type="checkbox" name="existing_project_yes" /> Sim</label>
      <label><input type="checkbox" name="existing_project_no" /> Não</label>
    </fieldset>
  </section>
  <section data-block="stack"></section>
  <section data-block="seguranca"></section>
  <section data-block="testes"></section>
  <section data-block="restricoes">
    <label>Restrições</label><textarea name="restricoes"></textarea>
  </section>
  <section data-block="observacoes">
    <label>Observações / Status</label>
    <textarea name="observacoes"></textarea>
    <input type="text" name="percentual_preenchido" placeholder="% respondido" />
    <label><input type="checkbox" name="status_ok" /> Questionário consistente</label>
    <label><input type="checkbox" name="status_pendente" /> Há pendências/gaps</label>
  </section>
</form>

```

A.10 Inteligência de validação do n8n sobre o questionário

O n8n do GCA deve executar validações de coerência entre respostas, apontar gaps, identificar dependências ausentes e devolver um parecer por projeto. Exemplos: marcar como gap a escolha de arquitetura event-driven sem mensageria; backend declarado sem persistência quando houver trilhas/auditoria; frontend autenticado sem mecanismo de autenticação; uso de IA externa sem política de restrição; melhoria em sistema existente sem repositório informado.

A.11 Compartimentalização, acesso e auditoria

O questionário, seus anexos, os resultados do n8n e as sugestões do GCA pertencem exclusivamente ao tenant/projeto que os originou. Outros tenants não podem visualizar, pesquisar ou herdar esses dados. O acesso é restrito aos integrantes do projeto conforme RBAC e ao Admin do GCA para coaching, suporte operacional e investigação de incidentes. Os logs de todos os projetos devem ser preservados na camada administrativa do GCA, com retenção, trilha append-only, filtros por projeto e mecanismos de investigação segura.

A.12 Campos finais obrigatórios

Pergunta / Campo	Resposta por seleção
50. Restrições	_____
51. Observações	_____
52. Percentual respondido	_____
53. Status do questionário	☐ OK para ingestão ☐ Pendente de ajustes ☐ Inconsistente - revisar
54. Áreas que validaram o questionário	☐ Negócio ☐ Arquitetura ☐ Desenvolvimento ☐ QA ☐ Segurança ☐ Infraestrutura ☐ Compliance